

Écrit pour Ruby 4.0 et Rails 8.1. Pour lire du code écrit pour une version plus ancienne : section 11.

Le code d'un fichier n'est pas toujours celui qui s'exécute : en cas de doute, interrogez le programme en console (section 3).

1. Lire du Ruby sans se tromper

Vous voyez	Ça veut dire
<code>if value</code>	seuls <code>nil</code> et <code>false</code> sont faux (truthiness) : <code>0</code> , <code>"</code> et <code>[]</code> sont vrais
<code>order.total</code>	toujours un appel de méthode, jamais un accès direct à un attribut ; les parenthèses sont facultatives
<code>def total ... end</code>	la méthode renvoie sa dernière expression, sans <code>return</code>
<code>empty?</code> , <code>save!</code> , <code>name=</code>	<code>?</code> renvoie un booléen, <code>!</code> signale la variante dangereuse (qui lève ou modifie en place), <code>=</code> définit un setter
<code>:draft</code> , <code>{ status: :draft }</code>	un symbole (une étiquette immuable), un hash à clés symboles
<code>def call(id, force: false, **options)</code>	un argument positionnel, un argument nommé avec sa valeur par défaut, les autres arguments nommés
<code>@total</code> , <code>@@count</code> , <code>MAX_ITEMS</code>	variable d'instance, variable de classe, constante
<code>items.map { item item.price }</code>	un bloc, passé à la méthode, qu'elle appelle avec <code>yield</code>
<code>map(&:price)</code> , <code>map { it.price }</code>	la même chose en raccourci (<code>it</code> depuis Ruby 3.4, <code>_1</code> avant)
<code>author&.name</code>	<code>nil</code> si <code>author</code> est <code>nil</code> , au lieu d'une erreur
<code>@total = compute_total</code>	calcule une fois, puis réutilise (mémoïsation)
<code>class String ... end</code> dans votre code	rouvre la classe existante et la modifie : les classes sont ouvertes

Enumerable, à connaître : `map`, `select`, `reject`, `find`, `any?`, `all?`, `sum`, `count`, `group_by`, `partition`, `tally`, `sort_by`, `min_by`, `each_with_object`, `each_slice`, `flat_map`.

2. La métaprogrammation à reconnaître

Une méthode générée ne laisse pas de `def` à trouver avec `grep`. Voilà ce qui en fabrique :

Vous voyez	Ce qui se passe
<code>attr_accessor :name</code>	génère <code>name</code> et <code>name=</code> (<code>attr_reader</code> et <code>attr_writer</code> : l'un ou l'autre)
<code>has_many</code> , <code>belongs_to</code> , <code>enum</code> , <code>scope</code> , <code>validates</code>	des macros : des méthodes de classe appelées au chargement, qui génèrent d'autres méthodes
<code>define_method(:name) { ... }</code>	définit une méthode à l'exécution, souvent dans une boucle sur une liste de noms
<code>method_missing(name, *args)</code>	intercepte les appels aux méthodes qui n'existent pas ; va avec <code>respond_to_missing?</code>

Vous voyez	Ce qui se passe
<code>send(:name, arg), public_send</code>	un appel dont le nom est calculé ; <code>send</code> ignore <code>private</code>
<code>include M, extend M</code>	ajoute les méthodes de <code>M</code> aux instances de la classe, ou à l'objet lui-même (souvent une classe)
<code>extend ActiveSupport::Concern, included do</code>	un concern : un module qui apporte à la fois des macros et des méthodes

Ordre de recherche d'une méthode : les méthodes propres à l'objet, puis sa classe, les modules qu'elle inclut (le dernier inclus d'abord), la classe parente, et ainsi de suite jusqu'à `BasicObject`. Si rien ne répond : `method_missing`.
`MyClass.ancestors` affiche cette chaîne.

♦ 3. Retrouver d'où vient une méthode

Besoin	Outil
Ce que sait faire un objet, groupé par module d'origine	<code>ls object</code> , filtré : <code>ls object -g total</code> (commandes IRB)
Le code d'une méthode, sans ouvrir de fichier	<code>show_source MyClass#my_method</code> , <code>show_source object.my_method</code>
Où est-elle définie ?	<code>object.method(:name).source_location</code>
Qui la définit dans la hiérarchie ?	<code>object.method(:name).owner</code> , <code>MyClass.ancestors</code>
Ce que la classe définit elle-même	<code>MyClass.instance_methods(false)</code>
La méthode existe-t-elle vraiment ?	<code>object.respond_to?(:name)</code> ; <code>false</code> alors que l'appel marche : <code>method_missing</code>
L'état d'un objet	<code>object.instance_variables</code> , <code>record.attributes</code> (Active Record)
M'arrêter ici et regarder	<code>binding.irb</code> (puis <code>whereami</code> , <code>exit</code>), ou <code>debugger</code> (gem <code>debug</code> : <code>next</code> , <code>step</code> , <code>continue</code>)
Le code d'une gem	<code>bundle info turbo-rails</code> (son chemin), <code>bundle open turbo-rails</code>

Pour une méthode qu'aucun `grep` ne trouve : elle est générée (`define_method`, une macro), servie par `method_missing`, ou apportée par un module (`include`, `extend`, un concern). Cherchez d'abord **ce qui génère**.

♦ 4. La console

```
bin/rails console      # console de développement
bin/rails console --sandbox # tout est annulé à la sortie : pour explorer sans risque
bin/rails runner script.rb # exécuter un script dans l'application
```

Dans la console	Effet
<code>reload!</code>	recharge le code de l'application après une modification
<code>_</code>	la dernière valeur renvoyée
<code>relation.to_sql</code>	le SQL d'une relation, sans l'exécuter
<code>app.host! "localhost"</code> , puis <code>app.get "/books"</code> et <code>app.response.status</code>	une requête HTTP à l'application (sans <code>host!</code> , l'hôte par défaut est bloqué en développement)
<code>helper.number_to_currency(1234.5)</code>	appeler un helper de vue
<code>ActiveRecord::Base.logger = Logger.new(\$stdout)</code>	afficher le SQL dans un script ou un <code>runner</code>

♦ 5. Où sont les choses

Le chemin du fichier donne le nom de la constante (Zeitwerk). `bin/rails zeitwerk:check` vérifie que tout le code respecte la règle.

Vous cherchez	Le fichier
Book	<code>app/models/book.rb</code> , table <code>books</code>
Archivable	<code>app/models/concerns/archivable.rb</code>
Admin::BooksController	<code>app/controllers/admin/books_controller.rb</code>
les colonnes d'un modèle	<code>db/schema.rb</code> , pas le fichier du modèle
la vue de <code>BooksController#show</code>	<code>app/views/books/show.html.erb</code> , rendue sans appel explicite
render <code>@books</code>	<code>app/views/books/_book.html.erb</code> , une fois par élément
<code>book_path(book)</code> , <code>books_url</code>	<code>config/routes.rb</code> : <code>resources :books</code> donne 7 actions et leurs helpers
une méthode disponible dans toutes les vues	<code>app/helpers/</code> , puis les helpers de Rails et des gems
ce qui s'exécute avant chaque action	les <code>before_action</code> du contrôleur et de <code>ApplicationController</code>
ce qui s'exécute au démarrage	<code>config/application.rb</code> , <code>config/initializers/</code> , les gems du Gemfile
les messages et les noms traduits	<code>config/locales/*.yml</code>

♦ 6. Lire un modèle Active Record

```
class Book < ApplicationRecord
  belongs_to :author                # author, author= ; obligatoire par défaut
  has_many :loans, dependent: :destroy # loans, loans.build ; détruits avec le livre
  has_many :members, through: :loans  # jointure en passant par loans
  has_many :reviews, as: :reviewable  # association polymorphe
  has_one_attached :cover              # un fichier joint (Active Storage)
  enum :status, { available: 0, on_loan: 1 } # on_loan?, on_loan!, Book.on_loan...
  normalizes :title, with: ->(title) { title.strip }
  scope :long, -> { where(pages: 500..) }
  validates :title, presence: true, uniqueness: { scope: :author_id }
  validate :cover_is_an_image         # une validation écrite dans une méthode
  before_validation :set_defaults     # à chaque valid?, save, create, update
  after_create :refresh_status         # juste après l'INSERT
end
```

`save` écrit tout de suite. Si une validation échoue, `save` et `update` renvoient `false`, `create` renvoie un objet non enregistré (`persisted?` vaut `false`); `save!`, `update!` et `create!` lèvent une exception. `delete`, `update_column` et `update_all` écrivent sans validations ni callbacks.

Ordre des callbacks à la création : `before_validation`, `after_validation`, `before_save`, `before_create`, l'INSERT, `after_create`, `after_save`, le COMMIT, `after_commit`. Quand une ligne change toute seule, cherchez un callback, sans oublier les `dependent:`.

```
Book.column_names
Book.reflect_on_all_associations.map { |a| [a.macro, a.name] }
Book.validators.map { |v| [v.kind, v.attributes] }
Book._save_callbacks.map { |c| [c.kind, c.filter] } # aussi _create_, _commit_callbacks...
Book.defined_enums
```

♦ 7. Diagnostiquer les requêtes

Besoin	Outil
Voir le SQL sans l'exécuter	<code>relation.to_sql</code>
Compter les requêtes d'une page	la fin du log de la requête : <code>ActiveRecord: 1.7ms (6 queries, 2 cached)</code>
Trouver d'où part une requête	la ligne <code>↳ app/...</code> sous chaque requête du log
Repérer un N+1	le même <code>SELECT</code> répété, avec un id différent à chaque ligne
Éviter le N+1	<code>includes(:author)</code> : une requête de plus, au lieu d'une par ligne
Compter	<code>count</code> (toujours une requête <code>COUNT</code>), <code>size</code> (utilise ce qui est déjà chargé)

Une relation est lazy : rien ne part tant qu'on n'a pas besoin du résultat, et chaque `where` ou `scope` renvoie une **nouvelle** relation.

Injection SQL : jamais de donnée reçue interpolée dans une chaîne SQL.

```
Book.where("title LIKE '%#{params[:q]}%") # injectable
Book.where("title LIKE ?", "%#{params[:q]}%") # sûr : Rails échappe la valeur
Book.where(author_id: params[:author_id]) # sûr
```

♦ 8. Contrôleurs, formulaires, Turbo et Stimulus

- Un contrôleur se lit dans l'ordre : les `before_action`, les actions publiques, puis sous `private` les méthodes utilitaires (`set_book`, `book_params`).
- Succès : `redirect_to`, avec `status: :see_other` (303) après un `DELETE` ou un `PATCH`. Échec : `render :new`, `status: :unprocessable_content` (422), sinon Turbo n'affiche pas le formulaire en erreur.
- Une action qui modifie des données passe par un formulaire (`form_with`, `button_to`), jamais par un simple lien.
- Turbo Drive, actif par défaut, envoie liens et formulaires en arrière-plan et remplace le `<body>`. `data: { turbo_confirm: "..."}` demande une confirmation.
- Turbo Frame : `turbo_frame_tag "nom"` délimite une zone. Un lien ou un formulaire placé dedans ne remplace que le frame de même id trouvé dans la réponse ; `data: { turbo_frame: "_top" }` pour en sortir.
- Turbo Stream : `format.turbo_stream` et un template `action.turbo_stream.erb` ; `turbo_stream.append`, `update`, `replace`, `remove` visent des éléments par leur id.
- D'un navigateur à l'autre : `turbo_stream_from @book` dans la vue, `broadcasts_refreshes_to :book` dans le modèle.
- Morphing : `turbo_refreshes_with method: :morph`, `scroll: :preserve` dans le `<head>` ; un rafraîchissement ne modifie que ce qui a changé et garde la position de défilement.
- Stimulus : un contrôleur par fichier, `app/javascript/controllers/counter_controller.js` (bin/rails generate stimulus counter), chargé sans déclaration. `data-controller="counter"` l'attache à un élément (`this.element`), `data-action="input->counter#update"` appelle `update()` à chaque événement `input`, `connect()` s'exécute quand l'élément apparaît.
- Targets : `static targets = [ "input" ]`, `data-counter-target="input"` dans le HTML, `this.inputTarget` dans le code. Si rien ne se passe, lisez la console du navigateur.

♦ 9. La ligne de commande

```
bin/rails routes -g book      # routes qui contiennent « book »
bin/rails routes -c books     # routes d'un contrôleur
bin/rails db:migrate          # migrations en attente
bin/rails db:migrate:status   # état de chaque migration
bin/rails db:rollback         # défaire la dernière
bin/rails db:seed             # rejouer les seeds
bin/rails db:reset            # recréer la base depuis db/schema.rb, puis les seeds
bin/rails generate model|migration|controller|scaffold ...
bin/rails destroy ...        # défaire un generate
bin/brakeman                  # analyse de sécurité du code
bin/bundler-audit             # failles connues des gems
```

♦ 10. Les tests

```
bin/rails test                # toute la suite
bin/rails test test/models    # un dossier
bin/rails test test/models/book_test.rb:12 # le test de la ligne 12
```

- `test "..."` définit un test; `setup do ... end` s'exécute avant chacun.
- Les fixtures (`test/fixtures/*.yml`) se lisent avec `books(:germinal)` ; une fixture en désigne une autre par son nom (`author: zola`).
- Modèle: `ActiveSupport::TestCase`, dans `test/models/`. Requête: `ActionDispatch::IntegrationTest`, avec `get`, `post`, `patch`, `delete`.
- Assertions: `assert`, `assert_not`, `assert_equal` attendu, obtenu, `assert_nil`, `assert_includes`, `assert_difference` "Loan.count" `do ... end`, `assert_no_difference`, `assert_response :success`, `assert_redirected_to`, `assert_select`.
- Un callback a modifié un autre objet que votre variable : `reload` relit la ligne en base.
- Un échec affiche `Expected` et `Actual`, puis la commande qui relance ce seul test.

♦ 11. Dans du code plus ancien

Rails 8.1	Plus ancien
<code>params.expect(book: [:title])</code>	<code>params.require(:book).permit(:title)</code>
<code>status: :unprocessable_content</code>	<code>status: :unprocessable_entity</code>
<code>enum :status, { ... }</code>	<code>enum status: { ... }</code>
<code>normalizes :email, with: ...</code>	<code>un before_validation</code>
<code>form.textarea</code> , <code>form.checkbox</code>	<code>form.text_area</code> , <code>form.check_box</code>
<code>ls</code> et <code>show_source</code> dans IRB	IRB récent requis, sinon la gem <code>pry</code> (<code>ls</code> , <code>show-source</code>)

♦ 12. Pour aller plus loin

- Rails Guides : guides.rubyonrails.org, et ceux d'une version précise : guides.rubyonrails.org/v7.1/ (remplacez le numéro)
- API : api.rubyonrails.org, rubyapi.org
- Turbo : turbo.hotwired.dev ; Stimulus : stimulus.hotwired.dev
- Du Rails idiomatique à lire : les dépôts ouverts de 37signals, github.com/basecamp
- Sandi Metz, *Practical Object-Oriented Design in Ruby*